Seminario de Automatización

Clase 4

APIs propias, límites de la plataforma y manejo de errores.

Tecnicatura Universitaria en Programación
Gonzalo Bechara Balaldi

De dónde venimos

Ya sabemos

- Modelar un proceso y llevarlo a un flujo
- Homogeneizar lo que producen distintos sistemas
- Recibir un aviso, validarlo y contestar
- Bifurcar con reglas explícitas

Hoy

- Le hablamos a cualquier servicio, tenga nodo o no
- Y dejamos de suponer que nos va a contestar

Hablarle a cualquier API

El nodo que sirve para todo

n8n trae cientos de nodos dedicados. Los servicios del mundo son muchos más.

Nodo dedicado (de marca)

Campos con nombre, credencial resuelta, operaciones listadas. Cuando existe, se usa.

Gmail, Slack, Google Sheets, Postgres.

HTTP Request

El nodo genérico. Cualquier cosa que hable HTTP entra por acá: la API de un proveedor, la de la empresa, la que escribió un compañero.

Todo lo demás.

El techo de la plataforma no es la lista de nodos. Es cualquier cosa que **no habla HTTP**, que hoy es muy poco.

Las cuatro partes de un pedido

- **Método.** Qué querés hacer. `GET` leer, `POST` crear, `PUT` y `PATCH` modificar, `DELETE` borrar.
- **URL.** A qué recurso. Puede llevar parámetros de consulta.
- **Cabeceras.** Cómo viene el cuerpo, quién sos, qué aceptás de vuelta.
- **Cuerpo.** El dato que mandás. Solo en los métodos que escriben.

```
POST /v1/paneles HTTP/1.1
Host: panel.ejemplo.com
Content-Type: application/json
Authorization: Bearer ****

{
  "panel": "cotizaciones-usd",
  "mercados": 6
}
```

Lo mismo que en los webhooks, pero ahora nosotros iniciamos la solicitud HTTP en vez de recibirla.

Autenticación por clave

En una cabecera

Lo más común. `Authorization: Bearer ...` o una cabecera propia del proveedor.

En la URL

Un parámetro de consulta con la clave. Cómodo y peor: queda escrito en todos los registros del camino.

(Usar cuando el token es de corta duración.)

OAuth2

Cuando el servicio actúa en nombre de un usuario.

(Se ve más adelante.)

La clave va en una **credencial** de n8n, nunca escrita dentro de un campo del nodo. Un nodo con la clave adentro es un nodo que no se puede exportar, ni versionar, ni compartir.

Qué es una credencial en n8n

Un objeto aparte, guardado cifrado, que el nodo **referencia por nombre**.

- `Dedicada.` El proveedor tiene su tipo propio: Gmail, Slack, Postgres.
- `Genérica.` *Header Auth*, *Query Auth*, *Basic Auth*. Para todo lo demás.

> La genérica de cabecera resuelve casi cualquier API con clave: nombre de la cabecera y valor.

```
Credencial: "GitHub del seminario"
Tipo:       Header Auth

  Name   →  Authorization
  Value  →  Bearer github_pat_11A...

El nodo solo dice:
  "usá la credencial GitHub del seminario"
```

Lo que viaja en el JSON exportado es el nombre. El valor se queda en la instancia.

Los códigos de respuesta HTTP, del lado del emisor

Código	Qué significa para vos
`2xx`	Ok - Salió. Igual hay que mirar el cuerpo.
`3xx`	Redirect - La URL que pusiste ya no existe (existió).
`400`	Bad request - Mandaste algo mal.
`401`	Unauthorized - Credencial ausente.
`403`	Forbidden - Credencial sin permiso.
`404`	Not found - Lo que buscás no existe. Puede ser tu URL.
`409`	Conflict - Algo que querés hacer ya está hecho.
`429`	Too many requests - Mandás solicitudes muy rápido. Esperá.
`5xx`	Internal server error - Se rompió algo del lado de ellos.

> **La línea que importa**
>
> Los `4xx` son tuyos: algo del pedido está mal y reintentar no lo arregla. (Excepto 429)

Qué hacer con la respuesta

El servicio contestó, el nodo está verde, y la mitad de las filas no se pueden usar. Tenemos que hacer lo que vimos las clases anteriores.

Un importe llega como texto

`"1015.00"`. No es un error del proveedor: es su contrato.

Falta una clave

La fila no trae fecha. Publicarla sin fecha es peor que no publicarla.

Un nulo legítimo

Esa casa no tiene precio de compra. Ausente no es inválido.

Un dato imposible

La venta es menor que la compra. Numéricamente válido, comercialmente no.

> Validar el **código de respuesta** y validar el **cuerpo de la respuesta** son dos cosas distintas, y hacen falta las dos.

Cuando falla

Qué puede salir mal



1 · Salir
No resuelve el nombre, no hay ruta, el certificado venció.(Le erraste)

2 · Llegar
El servicio está caído, saturado, o te está limitando.

3 · Volver
Tarda más que el tiempo de espera. (No sabemos si falló o no).

4 · Entender
Contestó, pero con una forma que tu flujo no esperaba.

El caso 3 es el peor: **no sabés si pasó o no pasó**. Por eso existe la idempotencia que vimos la clase pasada, ahora del lado del que llama.

Transitorio o permanente

Transitorio

Si lo intentás de nuevo en diez segundos, probablemente funcione.

`429`, `5xx`, tiempos de espera, cortes de red.

Lo arregla el reintento, sin molestar a nadie.

Permanente

Va a fallar igual las próximas mil veces.

URL mal escrita, credencial vencida, cuerpo que no cumple el contrato.

Reintentarlo son tres fallas en vez de una.

Reintentar

Se configura en el nodo, no se programa: **Settings → Retry On Fail**.

- `Cuántas veces.` Tres alcanza casi siempre.
- `Cuánto esperar.` Cinco segundos entre intentos.

El reintento dispara con **cualquier** error, no solo con los transitorios. No hay filtro por código.

Va primero

Antes de construir ramas de error, absorbé lo transitorio. Así la rama de error dispara solo cuando el problema es real.

Cuidado al escribir

Reintentar un `GET` es gratis. Reintentar un `POST` que crea algo puede crearlo dos veces, salvo que el otro lado sepa descartar duplicados.

La salida de error son dos pasos

Es la forma más común de "manejar" errores sin manejarlos.

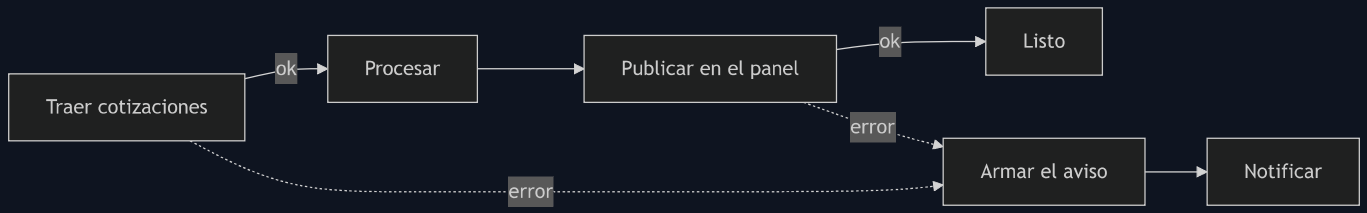
1	En el nodo: `Settings → On Error → Continue (using error output)`. Eso `crea` la segunda salida.	habilitar
2	Cablear esa salida a algún lado que haga algo con el error.	conectar

Solo el 1
El error se descarta en silencio y la ejecución figura como exitosa. El peor de los tres.

Solo el 2
La salida nunca dispara. El flujo se corta como si no hubiera nada.

Los dos
El error baja por la segunda salida hasta tu rama.

Una sola rama de aviso



Todas las salidas de error al mismo lugar

El lienzo queda legible y la lógica del aviso se escribe una sola vez.

El aviso dice qué pasó

Qué nodo, qué código, si conviene reintentar, y qué mirar. Un aviso que dice "falló el flujo" no sirve para nada.

El registro de ejecuciones

Cuando algo falla en producción, esto es lo único que tenés.

Entrada y salida de cada nodo

Con qué llegó y con qué se fue. Ahí se ve dónde se rompió la forma del dato.

Cuántos intentos hubo

Si el reintento estaba puesto, se ve cada uno.

El error completo

Código, cuerpo de la respuesta y nodo. Leer el cuerpo antes que el código.

Manual o de producción

Filtrar por eso ahorra la mitad del trabajo.

Las ejecuciones **no son eternas**. La instancia las va borrando. Si algo importa de verdad, se guarda en otro lado, no en el registro.

Caso de uso

Un panel de cotizaciones

Una API pública publica las cotizaciones del dólar. Hay que armar un panel con eso y dejarlo publicado en algún lado que se pueda abrir.

El origen · sin credencial

Una API pública, gratuita y abierta. Devuelve una lista de casas de cambio con compra, venta y fecha.

Cualquiera la consulta. Nadie sabe quién sos.

El destino · con credencial

La API de GitHub. El panel se publica como un archivo con su propia dirección, que se abre en el navegador.

Escribir exige decir quién sos.

Ninguna de las dos tiene nodo dedicado para lo que necesitamos. Sin el nodo genérico, este flujo no existe.

No es casualidad que la solicitud de escritura sea la que pide credencial. Leer algo público es gratis; dejar algo escrito, no.

Qué hay que producir

1	Traer las cotizaciones y `mirar` lo que vino	lectura
2	Separar lo publicable de lo que no, con el motivo escrito	validación
3	Traducir al vocabulario del panel	mapeo
4	Publicar `una sola vez`, autenticado, con todo junto	escritura
5	Una rama de error que avise qué pasó y si conviene reintentar	fallas

Práctica guiada

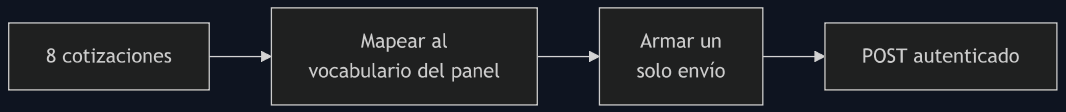
Paso 0 — Obtener nuestro token (uno por persona)

a	En GitHub: Settings → Developer settings → `Fine-grained tokens`	dos minutos
b	Un solo permiso: `Account → Gists → Read and write`. Nada más	mínimo
c	En n8n, credencial `Header Auth`: `Authorization` / `Bearer <token>`	una vez

Paso 1 — Traer y mirar

a	Importar el template. El nodo de entrada ya está armado	listo
b	Ejecutar y abrir la salida: ocho filas, con sus campos	observar
c	Recorrerlas de a una y anotar cuáles no se podrían publicar	a mano

Paso 2 — Mapear y publicar



Sin el nodo que junta, el HTTP se ejecuta **una vez por item** y crea seis publicaciones de una fila cada una. Es la lección de los items de la clase 1, y hoy además gasta seis veces la cuota.

Dos cabeceras más

El formato que aceptás y la versión de la API. Ninguna de las dos es la credencial.

Cómo saber que salió bien

La respuesta trae una dirección. Abrila: ahí está tu panel.

Paso 3 — La rama de error

a	En los dos nodos HTTP: On Error → Continue (using error output)	habilitar
b	Cablear las dos salidas de error al mismo nodo de aviso	conectar
c	Armar el aviso: qué nodo, qué código, transitorio o permanente	describir
d	Reintento en los dos: tres intentos, cinco segundos	absorber

> El aviso tiene que servirle a alguien que **no estaba mirando**. Si dice "error en el flujo", no sirve.

Paso 4 — Romperlo a propósito

Con el flujo andando, provocar tres fallas distintas y mirar el registro cada vez.

Un carácter menos en el token

Editar la credencial y romperla. `401`. ¿Cuántas veces reintentó antes de rendirse, y para qué sirvió?

Un tiempo de espera de 1 ms

Forzar que el pedido no llegue a volver. ¿Se publicó el panel del otro lado o no?

Un campo renombrado

Cambiar `venta` por `precio` en la validación. El flujo no falla y el resultado queda mal.

En la tercera: esposible que **ningún nodo se ponga en rojo** y el panel queda publicado con datos incorrectos. No todas las fallas son errores.

Lo que probablemente falle

`401` con el token recién creado

Token mal pegado en la credencial, o se copió con un espacio al final.

`403` al publicar

El token está bien pero le falta el permiso de escritura. Autenticado no es autorizado.

Se publicaron seis paneles

Falta juntar los items antes del HTTP. Un envío por item.

La rama de error nunca disparó

Está cableada pero falta el On Error. O está el On Error y no se cableó.

El aviso llegó vacío

Se buscó el error en la raíz del item. Viene en `$json.error`.

Cierre

Hoy vimos

- El nodo HTTP como comodín para cualquier servicio
- Credenciales: dónde va la clave y dónde nunca
- Transitorio contra permanente, y qué reintentar
- Que la salida de error son dos pasos, no uno
- Que una falla silenciosa es peor que un nodo en rojo